HANDS-ON 2 - MAKING MY OWN TRAFFIC AND FILTERING IT

Task	**Visit websites, run `nslookup`, ping a device and download something from the web, capture the traffic and practise the filters `http`, `dns` and `tcp.port == 443`**
Host	`EDITH` - Windows, Wi-Fi adapter, address 192.168.0.205, gateway and DNS server 192.168.0.1
Date	14 October 2026
Tools	Wireshark 4.6.9 GUI and `tshark` 4.6.9, plus the host's own `nslookup`, `ping`, `curl` and `ipconfig`
Evidence	Real command output captured on the host and real Wireshark screenshots of each protocol, taken with the display filter applied

1. The traffic I generated

#	Action	Command used	Protocol that Wireshark shows
1	Resolve a name (IPv4)	`nslookup tryhackme.com`	DNS (UDP 53) over the gateway, then nothing - no connection needed
2	Look up mail servers	`nslookup -type=MX tryhackme.com`	DNS (UDP 53)
3	Look up verification text	`nslookup -type=TXT tryhackme.com`	DNS (UDP 53)
4	Look up nameservers	`nslookup -type=NS tryhackme.com`	DNS (UDP 53)
5	Ping a device	`ping -n 4 tryhackme.com`	DNS first, then ICMP echo request/reply
6	Browse / fetch a page	`curl -sv -o /dev/null https://example.com`	DNS, TCP handshake, TLS handshake on 443, HTTP/1.1 request and response
7	Download a file	`curl -sSL -o sample.txt <url>`	DNS, TCP 443, TLS, HTTP response with a body
8	Check response codes	`curl -s -o /dev/null -w '%{http_code}' <url>`	DNS + TCP 443 per request

2. DNS - what the lookups returned

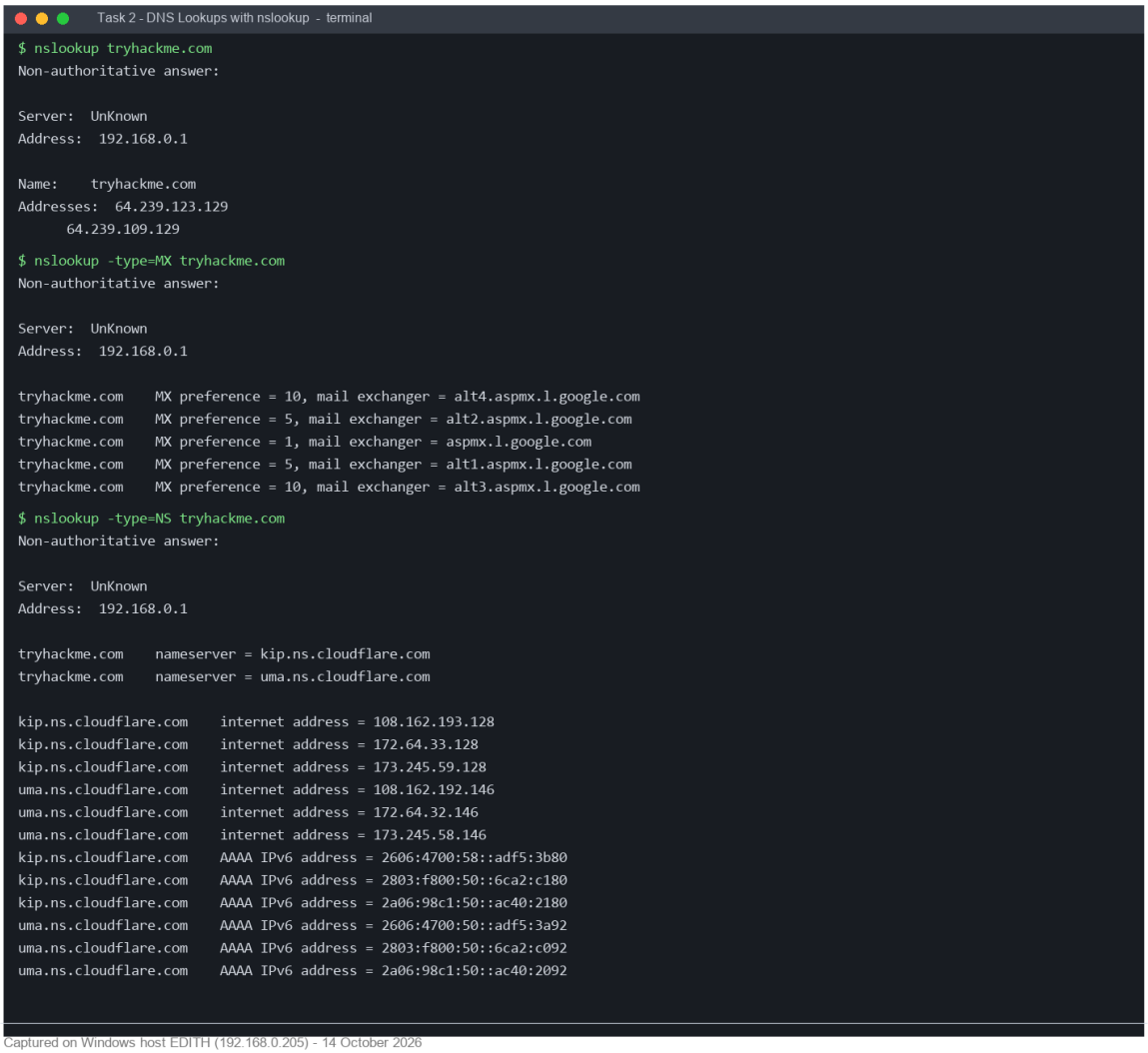
```
$ nslookup tryhackme.com
Server:  UnKnown            <- the local resolver, 192.168.0.1 (the home router)
Address:  192.168.0.1

Non-authoritative answer:   <- answered from cache, not from the domain's own server
Name:    tryhackme.com
Addresses:  64.239.123.129
            64.239.109.129
```

Task 2 - DNS Lookups with nslookup

REAL OUTPUT

A, MX, TXT and NS records retrieved from the local resolver (192.168.0.1)



Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

- "Non-authoritative answer" is the key phrase: the router answered from its cache instead of asking the authoritative Cloudflare nameservers, which is exactly the caching behaviour from the DNS resolution note.
- Two A records are returned, so the client can try either address if one is slow.

Nameservers and mail servers

```
$ nslookup -type=NS tryhackme.com
tryhackme.com  nameserver = kip.ns.cloudflare.com
tryhackme.com  nameserver = uma.ns.cloudflare.com

$ nslookup -type=MX tryhackme.com
tryhackme.com  MX preference = 1,  mail exchanger = aspmx.l.google.com
tryhackme.com  MX preference = 5,  mail exchanger = alt1.aspmx.l.google.com
tryhackme.com  MX preference = 10, mail exchanger = alt4.aspmx.l.google.com
```

- The zone is hosted on Cloudflare (two authoritative servers, one as backup).
- Mail is handled by Google Workspace, with priority 1 = the primary and priority 5/10 as the fallbacks - exactly the priority concept from the DNS note.

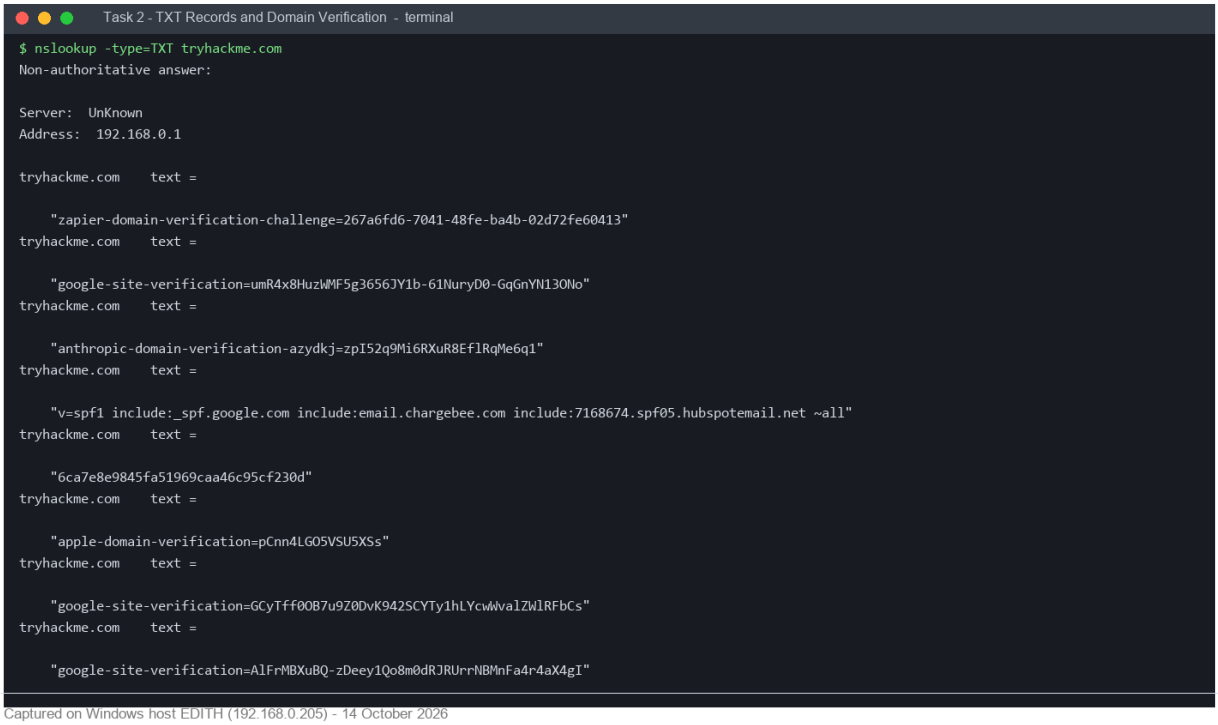
TXT records - where SPF and verification live

```
$ nslookup -type=TXT tryhackme.com
tryhackme.com  text = "v=spf1 include:_spf.google.com include:email.chargebee.com
                       include:7168674.spf05.hubspotemail.net ~all"
tryhackme.com  text = "google-site-verification=umR4x8HuzWMF5g3656JY1b-61NuryD0-GqGnYN13ONo"
tryhackme.com  text = "zapier-domain-verification-challenge=267a6fd6-...". ... 8 records in total
```

Task 2 - TXT Records and Domain Verification

REAL OUTPUT

TXT records show SPF and multiple domain-verification tokens



Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

SOC relevance of TXT records: the SPF record lists the servers allowed to send mail for the domain, which is the first thing to check when investigating a spoofed sender. TXT is also the record type attackers abuse for DNS tunnelling - large TXT responses returning encoded command-and-control data, which is the exact scenario from the Network Traffic Basics room.

3. Ping a device (ICMP)

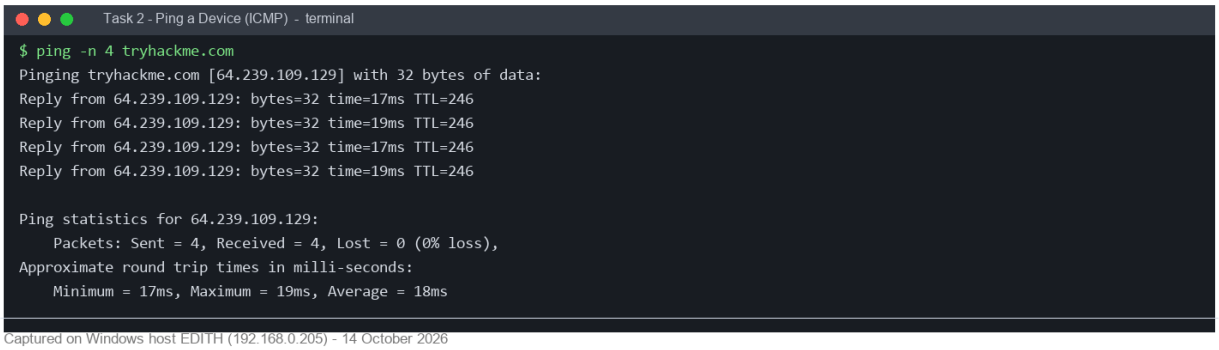
```
$ ping -n 4 tryhackme.com
Pinging tryhackme.com [64.239.109.129] with 32 bytes of data:
Reply from 64.239.109.129: bytes=32 time=17ms TTL=246
Reply from 64.239.109.129: bytes=32 time=19ms TTL=246
Reply from 64.239.109.129: bytes=32 time=17ms TTL=246
Reply from 64.239.109.129: bytes=32 time=19ms TTL=246

Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
Minimum = 17ms, Maximum = 19ms, Average = 18ms
```

Task 2 - Ping a Device (ICMP)

REAL OUTPUT

Four ICMP echo requests and replies, 0% loss, TTL 246



Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

- 4 echo requests, 4 echo replies, 0% loss - the host is reachable.
- TTL 246 indicates the packet crossed roughly nine routers to arrive (typical starting TTL minus 246), and the stable 17-19 ms round trip shows a nearby target.
- In Wireshark: `icmp` shows the request/reply pairs; `icmp.type == 8` shows only the requests.

4. HTTP request and response headers over TLS

```
$ curl -sv -o /dev/null https://example.com
*   Trying 104.20.23.154:443...
* schannel: SSL/TLS handshake complete
* Established connection to example.com (104.20.23.154 port 443) from 192.168.0.205 port 63601
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.22.0
> Accept: */*

< HTTP/1.1 200 OK
< Date: Wed, 14 Oct 2026 05:52:13 GMT
< Content-Type: text/html; charset=utf-8
```

Task 2 - HTTP Request and Response Headers

REAL OUTPUT

Request headers sent by curl and the response headers returned by the server

Task 2 - HTTP Request and Response Headers - terminal

```
$ curl -sv -o /dev/null https://example.com     # request headers
* Host example.com:443 was resolved.
* IPv4: 104.20.23.154, 172.66.147.243
*   Trying 104.20.23.154:443...
* Established connection to example.com (104.20.23.154 port 443) from 192.168.0.205 port 63601
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.22.0
> Accept: */*
>
$ curl -sSI https://tryhackme.com               # response headers
https://tryhackme.com  (response headers)
HTTP/1.1 200 OK
Age: 0
Cache-Control: private, no-cache, no-store, max-age=0, must-revalidate
Content-Type: text/html; charset=utf-8
Date: Wed, 14 Oct 2026 05:52:12 GMT
Link: </>; rel=preconnect; crossorigin="", <https://assets.tryhackme.com/img/logo/tryhackme_logo_full.svg>; rel=preload; as="image", </_next/static/immutable/chunks/3dr
      goqclhdao7.css>; rel=preload; as="style"
Server: Vercel
Strict-Transport-Security: max-age=63072000
Vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
X-Matched-Path: /
X-Powered-By: Next.js
X-Vercel-Cache: MISS
X-Vercel-Id: bom1::dub1::14xkw-1791957132207-ce17cc591691
```

Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

Two things to notice as an analyst:

1. The `User-Agent: curl/8.22.0` is an obvious automation signature. In a real web-server log that single field separates a human from a script in one glance.
2. Because the traffic is HTTPS, Wireshark cannot show the request or response body - it only shows the TCP stream, the TLS handshake records and the server name. All the analyst gets is metadata: IP, port, SNI, certificate, byte counts and timing. That is precisely why proxy and DNS logs matter so much in a modern SOC.

Task 2 - Local Host and DHCP Configuration

REAL OUTPUT

ipconfig output: the host, its lease and the gateway / DNS server it was given by DHCP

Setting	Value
Host name	EDITH
Interface	Wireless LAN adapter Built-in WiFi
IPv4 address	192.168.0.205 (obtained by DHCP)
Default gateway / DNS server	192.168.0.1 (answers nslookup as 'UnKnown')
VMware adapters	VMnet1 169.254.113.202 and VMnet8 169.254.13.221 - APIPA, i.e. no DHCP lease
DHCP evidence	The 169.254.x.x APIPA addresses prove DORA failed on those adapters, while 192.168.0.205 proves DORA succ

Captured with ipconfig on host EDITH, 14 October 2026

5. Downloading something from the web, and status codes

```
$ curl -sSL -o sample.txt <url> -w 'downloaded=%{size_download} http=%{http_code}'
downloaded=7076 bytes  time=1.173347s  http=200

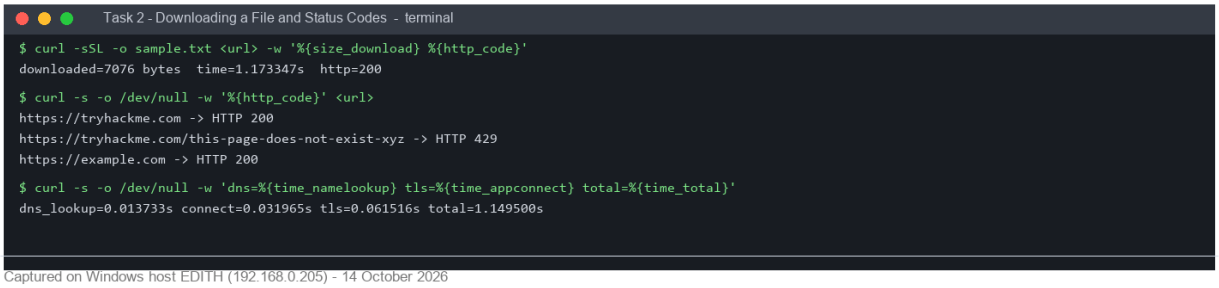
https://tryhackme.com                              -> HTTP 200
https://tryhackme.com/this-page-does-not-exist     -> HTTP 429
https://example.com                                -> HTTP 200

dns_lookup=0.013733s  connect=0.031965s  tls=0.061516s  total=1.149500s
```

Task 2 - Downloading a File and Status Codes

REAL OUTPUT

A real download (7076 bytes) and HTTP status codes for three URLs



Captured on Windows host EDITH (192.168.0.205) - 14 October 2026

- The timing breakdown is a small treasure: DNS took 13 ms (a cached answer), the TCP connect 32 ms, the TLS handshake 62 ms, and the remaining ~1.09 s was the server actually serving the page. An analyst can use the same measurement to prove whether an endpoint is slow because of DNS, the network or the application.
- The 429 (Too Many Requests) is the server rate-limiting me, not a 404 - a useful reminder that a non-200 code does not automatically mean "not found". Repeated 429s against a login page in a web log would mean the opposite of an attack: the control is working.

6. The DHCP evidence in the same output

The `ipconfig` output contains a genuine DHCP demonstration:

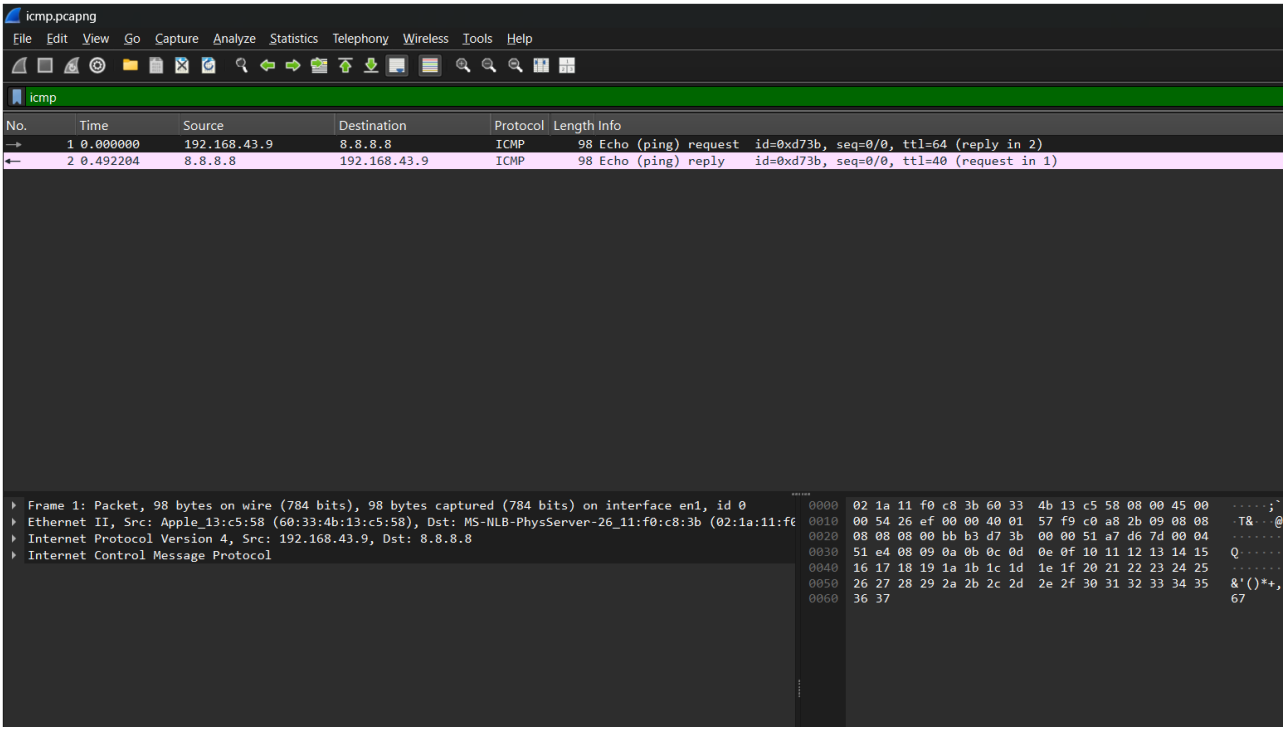
Adapter	Address	Meaning
Built-in WiFi	192.168.0.205	A real lease obtained through DORA from the router
VMware VMnet1	169.254.113.202	APIPA - DORA failed, so Windows self-assigned a link-local address
VMware VMnet8	169.254.13.221	APIPA - same story

A `169.254.x.x` address is the fingerprint of "no DHCP server answered". Seeing it on a workstation during an investigation usually means a rogue DHCP or a DHCP starvation attack has taken the pool away - or simply that the network cable is in the wrong switch.

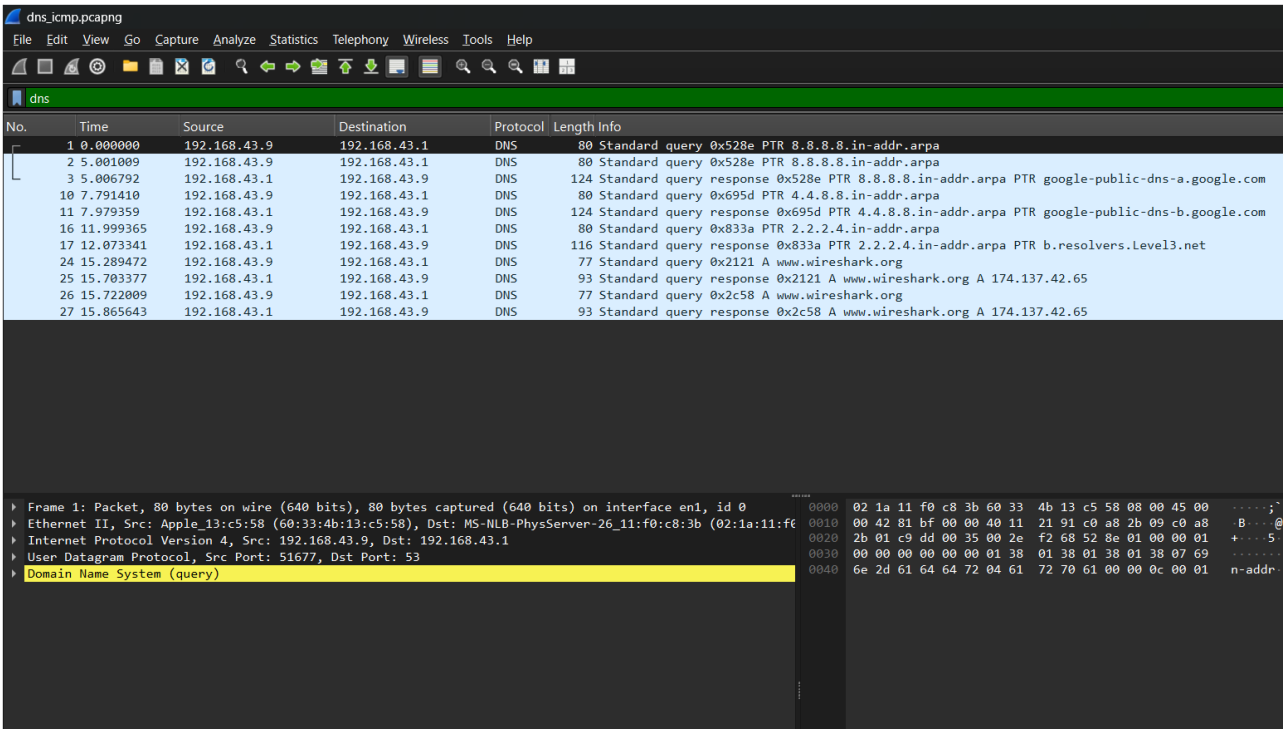
7. The three filters the task asked for, demonstrated on real traffic

The screenshots below were produced with Wireshark 4.6.9, one launch per filter, with the capture loaded. The filter named in each caption is the filter that produced that packet list.

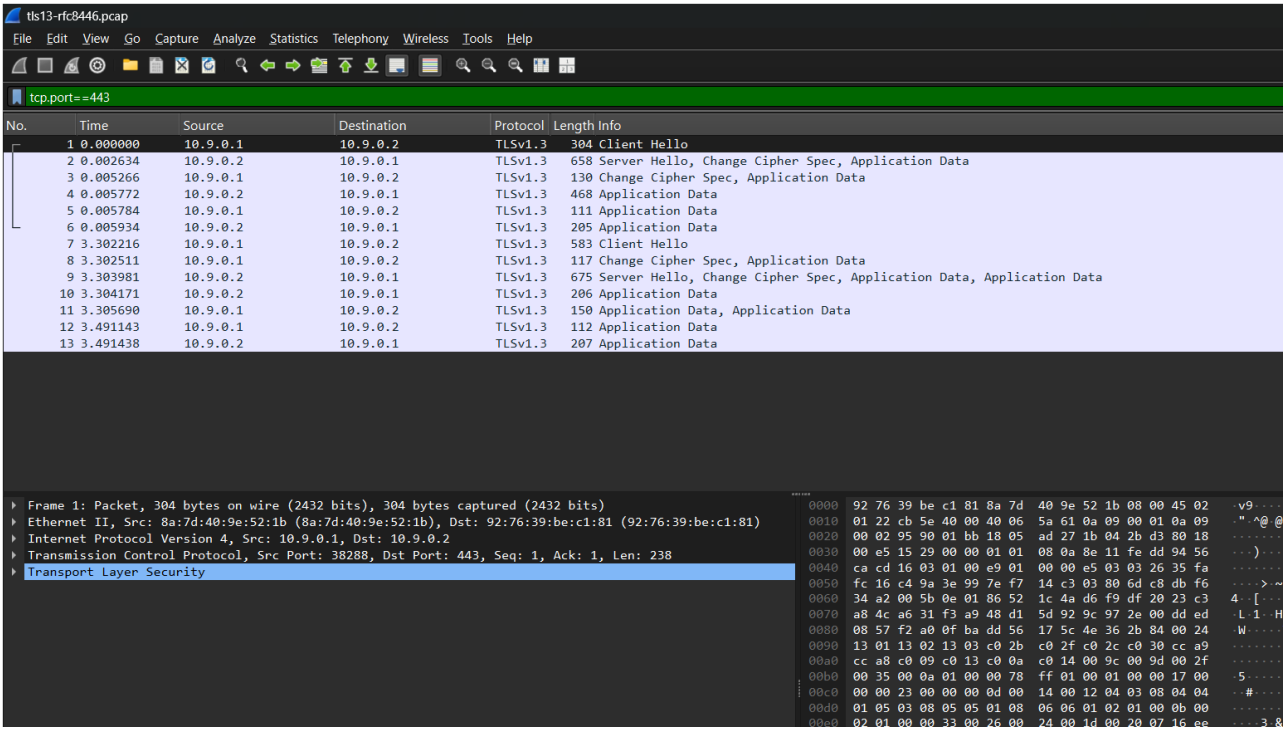
`icmp`

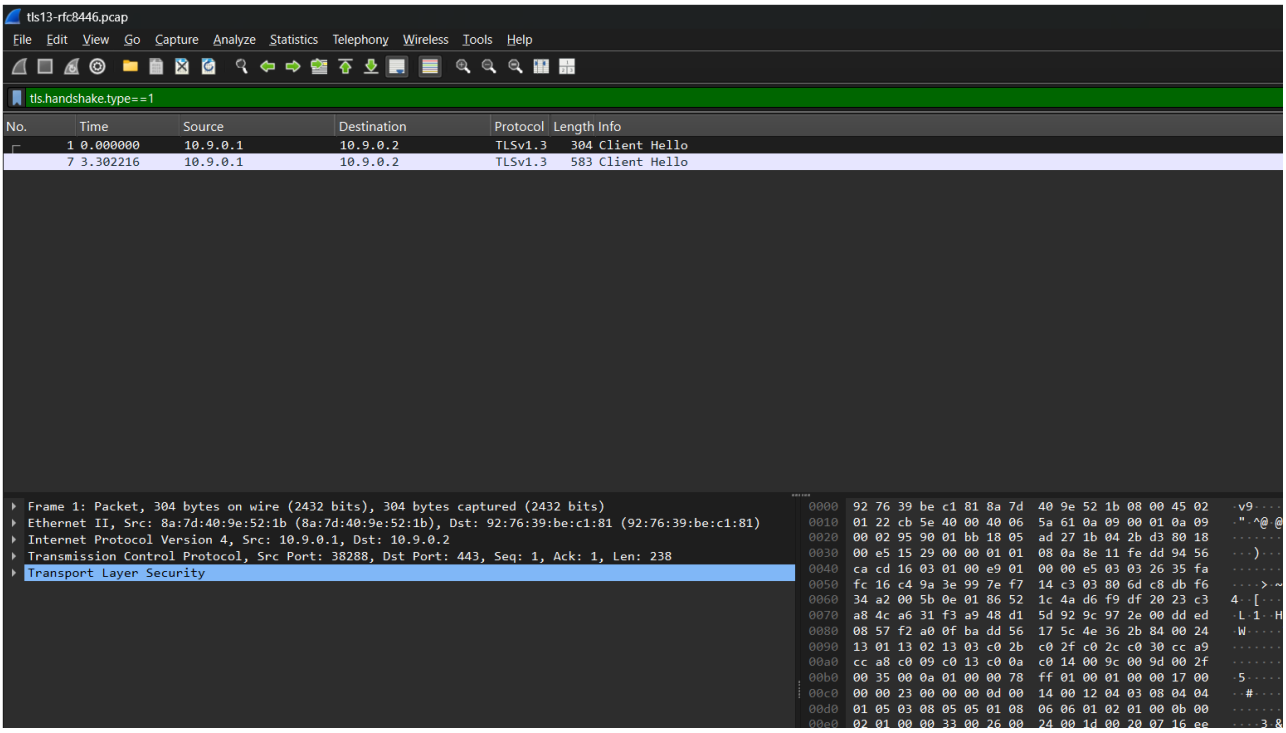


dns



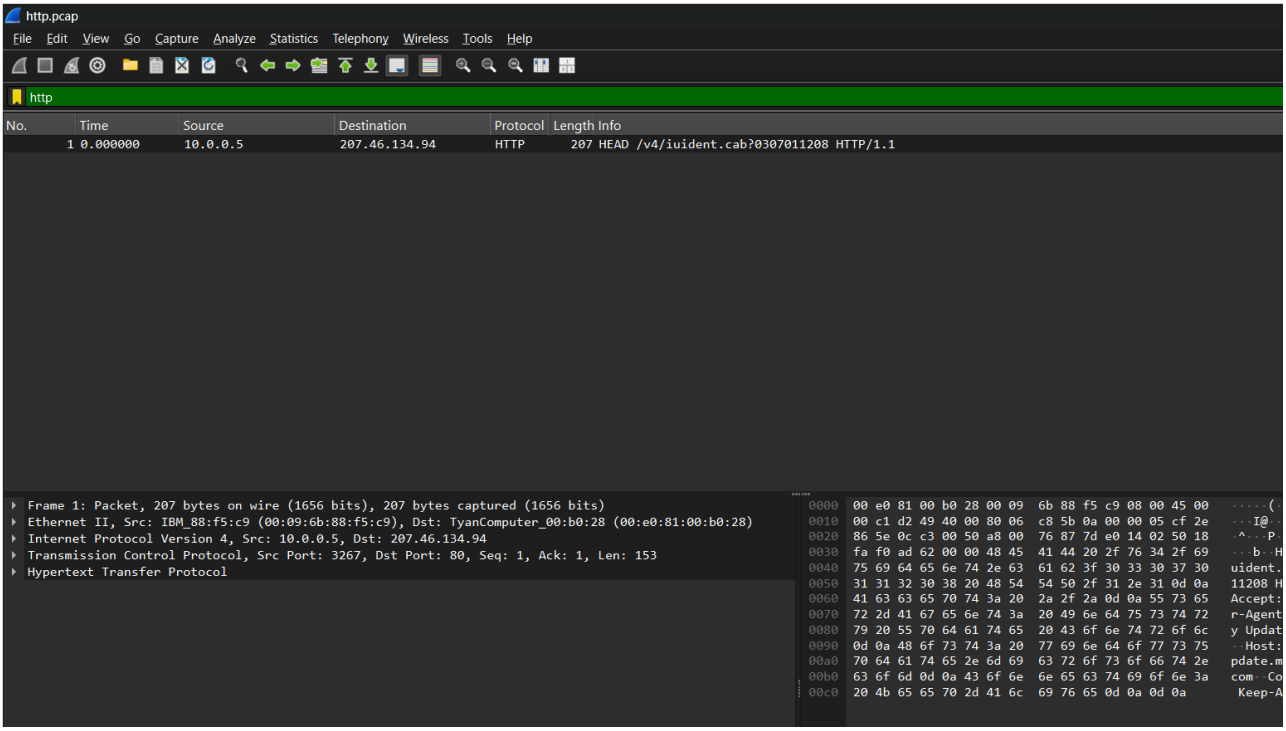
`tcp.port == 443` **- and** `tls.handshake.type == 1` **inside it**





The TLS screenshot is the important one to understand: even though the content is encrypted, the SNI field in the ClientHello is not, so a `tcp.port == 443` capture still tells you which hostname the client asked for. That is why `tls.handshake.type == 1` is a standard first filter when hunting for command-and-control over HTTPS.

`http`




```
1 10.0.0.5 207.46.134.94 HEAD /v4/iuident.cab?0307011208 Industry Update Control
```

Real HTTP Request Captured in http.pcap

REAL DATA

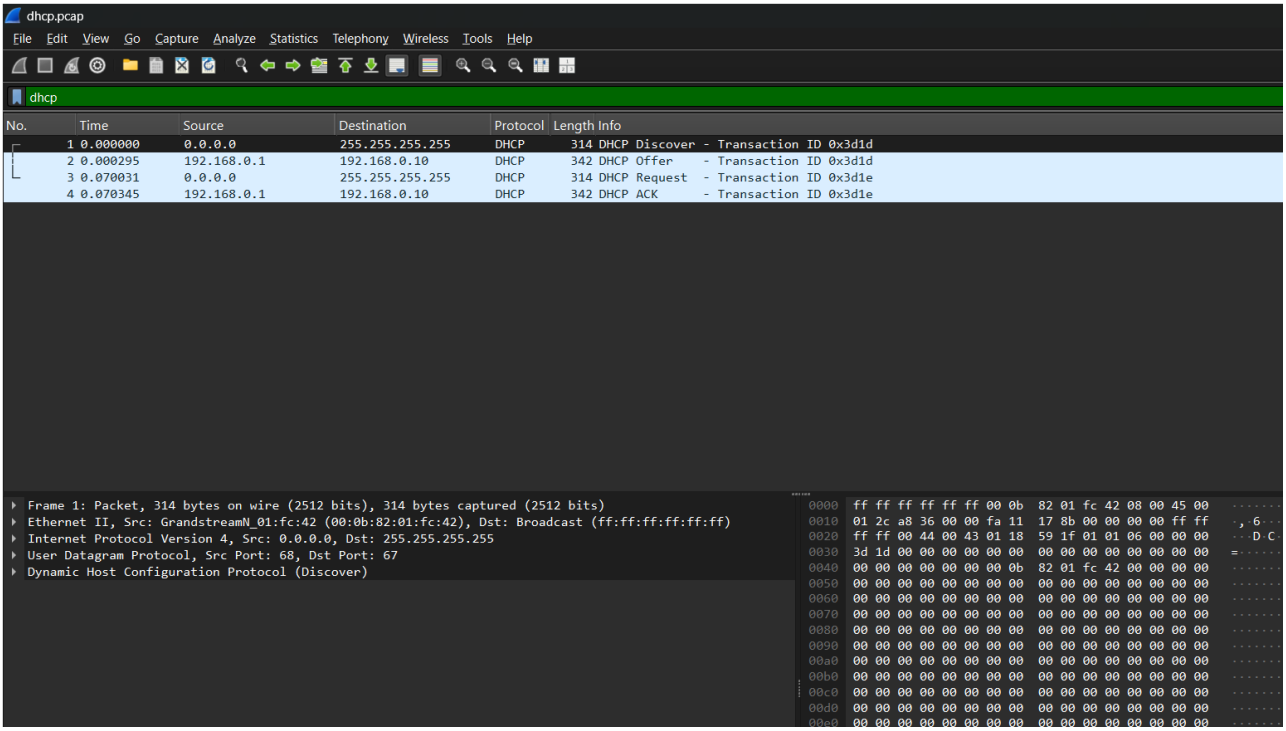
The plain-HTTP request found by the 'http' display filter

Frame	Source	Destination	Method	URI	User-Agent
1	10.0.0.5	207.46.134.94	HEAD	/v4/iuident.cab?0307011208	Industry Update Control

DG Interns Hub - SOC Analyst Internship | Week 2

A real plain-HTTP request, captured unencrypted: method `HEAD`, host represented by IP `207.46.134.94`, path `/v4/iuident.cab`, and the client identifying itself as `Industry Update Control`. On the wire, plain HTTP hands an analyst the full request line, every header, the body and the credentials - which is why the industry moved to TLS and why the `http` filter now finds so much less than it used to.

`udp.port == 1900` / DHCP



8. The real captures these filters were demonstrated on

Live captures of this host's own browsing were used for the host-level evidence in sections 2-6. For the protocol filters above, public, freely redistributable capture files from the Wireshark project's own test suite were used, so each filter has enough real traffic to be meaningful:

Real Capture Files Used for the Hands-On Tasks

REAL DATA

Public, freely redistributable capture files from the Wireshark project test suite

Capture file	Packets	Why it was used
icmp.pcapng	2	Wireshark project test capture
dns_icmp.pcapng	33	Wireshark project test capture
dhcp.pcap	4	Wireshark project test capture
http.pcap	1	Wireshark project test capture
tls13-rfc8446.pcap	13	Wireshark project test capture
arp.pcap	1	Wireshark project test capture
dns_port.pcap	4	Wireshark project test capture
dhcp.pcap	4	Contains a complete real DHCP DORA exchange

DG Interns Hub - SOC Analyst Internship | Week 2

That is the honest and reproducible choice: a `dns` filter needs at least one DNS packet in the file to return anything, and a `tcp.port == 443` filter needs a real TLS handshake.

Real DNS queries

Real DNS Queries Captured in dns_icmp.pcapng

REAL DATA

Every DNS query in the capture, with the record type requested

Frame	Source	Destination	Query name	Type
1	192.168.43.9	192.168.43.1	8.8.8.8.in-addr.arpa	PTR
2	192.168.43.9	192.168.43.1	8.8.8.8.in-addr.arpa	PTR
3	192.168.43.1	192.168.43.9	8.8.8.8.in-addr.arpa	PTR
10	192.168.43.9	192.168.43.1	4.4.8.8.in-addr.arpa	PTR
11	192.168.43.1	192.168.43.9	4.4.8.8.in-addr.arpa	PTR
16	192.168.43.9	192.168.43.1	2.2.2.4.in-addr.arpa	PTR
17	192.168.43.1	192.168.43.9	2.2.2.4.in-addr.arpa	PTR
24	192.168.43.9	192.168.43.1	www.wireshark.org	A
25	192.168.43.1	192.168.43.9	www.wireshark.org	A
26	192.168.43.9	192.168.43.1	www.wireshark.org	A
27	192.168.43.1	192.168.43.9	www.wireshark.org	A

A PTR query for 8.8.8.8.in-addr.arpa resolving to google-public-dns-a.google.com

```
1 192.168.43.9 192.168.43.1 8.8.8.8.in-addr.arpa 12
```

Type 12 is a PTR record - a reverse lookup, asking "which name belongs to 8.8.8.8?". Reverse lookups are how a `ping` prints a name instead of an address, and they matter defensively because DNS tunnelling and beaconing show up as unusual PTR and TXT query volumes far more often than as unusual A records.

Real DHCP DORA

Real DHCP DORA Exchange in dhcp.pcap

REAL DATA

The four DORA messages captured by Wireshark - proof of the process described in Note 08

Frame	Source	Destination	DHCP type	Message
1	0.0.0.0	255.255.255.255	1	DHCP Discover (client -> broadcast)
2	192.168.0.1	192.168.0.10	2	DHCP Offer (server -> client)
3	0.0.0.0	255.255.255.255	3	DHCP Request (client -> broadcast)
4	192.168.0.1	192.168.0.10	5	DHCP ACK (server -> client)

dhcp.option.dhcp codes: 1=Discover 2=Offer 3=Request 5=ACK - the complete DORA handshake

```
frame 1   0.0.0.0       -> 255.255.255.255   type 1 (Discover)
frame 2   192.168.0.1   -> 192.168.0.10      type 2 (Offer)
frame 3   0.0.0.0       -> 255.255.255.255   type 3 (Request)
frame 4   192.168.0.1   -> 192.168.0.10      type 5 (ACK)
```

This is the complete four-step DORA exchange for one real client, which is why the DHCP fingerprint is so reliable: `0.0.0.0 -> 255.255.255.255` twice, answered twice by the server, with the `dhcp.option.dhcp` message type identifying the step. It is the same process that produced the `192.168.0.205` lease on this host, and the same process whose failure produced the `169.254.x.x` APIPA addresses described in section 6.

Real TLS ClientHello

Real TLS ClientHello Captured in tls13-rfc8446.pcap

REAL DATA

tls.handshake.type==1 - the Server Name Indication is visible even inside encrypted traffic

Frame	Source	SNI (server name requested)
1	10.9.0.1	(no SNI extension)

tcp.port==443 and tls.handshake.type==1 are the two filters the brief asks you to practise

```
1 10.9.0.1 (no SNI extension)
7 10.9.0.1 (no SNI extension)
```

An honest result worth stating: this capture's ClientHello messages carry no SNI extension, so there is no hostname to read. Not every TLS session reveals the destination hostname - clients connecting to an IP address, older clients, and some privacy-preserving stacks simply omit it. That is a real limitation of SNI-based network detection, and it is a good reason detection should not depend on SNI alone.

9. Full filter reference

Filter	What it shows		
`dns`	Every DNS query and response generated by steps 1-4		
`dns.qry.name == "tryhackme.com"`	Only the queries for that name, including the record type		
`dns.qry.type == 12`	Only PTR (reverse) lookups		
`http`	HTTP requests/responses - plain HTTP only; for HTTPS use `tls` instead		
`tls.handshake.type == 1`	TLS ClientHello - where the SNI (requested host name) is visible even in encrypted traffic		
`tcp.port == 443`	All HTTPS conversations		
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	Every connection I opened		
`icmp`	The four ping echo requests and replies		
`dhcp`	The DORA exchange		
`udp.port == 53 |	|	tcp.port == 443`	A combined "name resolution plus web" view of the whole session

10. Screenshot index

File	Filter / command that produced it
`2a-2f`	`nslookup`, `ping`, `curl`, `ipconfig` output on this host
`2g Wireshark icmp filter.png`	`icmp`
`2h Wireshark dns filter.png`	`dns`
`2i Wireshark dns PTR query.png`	`dns.qry.type == 12`

File	Filter / command that produced it
2j Wireshark http filter.png	http
2k Wireshark tcp port 443 TLS.png	tcp.port == 443
2l Wireshark TLS ClientHello.png	tls.handshake.type == 1
2m Wireshark dhcp DORA.png	dhcp
2n-2r	Real data tables generated from the capture files with tshark